# OBSERVACIONES DE LA PRÁCTICA

David Gracia Arevalo, 202610322
Paula Gabriela Arias Sarmiento, 202616382
Luciana Caicedo Arroyo, 202511309

**Datos de la ejecución**

Se cargaron **319073 crímenes** del conjunto de datos Boston Crimes. Los resultados del árbol rojo-negro (RBT) y del árbol binario de búsqueda (BST) del laboratorio anterior se comparan en la siguiente tabla.

| **Métrica** | **BST (Lab 7)** | **RBT (Lab 8)** |
|---|---|---|
| Altura del árbol por fechas | 30 | 14 |
| Elementos en el árbol por fechas | 1177 | 1177 |
| Altura del árbol por áreas | 20 | 13 |
| Elementos en el árbol por áreas | 879 | 879 |
| Tiempo de carga (ms) | 14455.53 | 14316.39 |

Los tiempos dependen del equipo, del estado del sistema y de la implementación utilizada. Por esta razón, deben interpretarse como una referencia experimental y no como valores absolutos.

**a. ¿Qué diferencia existe entre las alturas de los dos árboles (BST y RBT)?**

La altura del RBT es considerablemente menor que la del BST para los mismos datos. Esto ocurre porque el RBT se mantiene balanceado mediante rotaciones y cambios de color después de las inserciones.

En un BST tradicional, la altura depende completamente del orden en que se insertan las llaves. Si las llaves llegan en un orden desfavorable, el árbol puede quedar desbalanceado y acercarse a una estructura lineal. En ese caso, la altura puede llegar a ser del orden de (N).

Por el contrario, el RBT mantiene una altura acotada por una función logarítmica

Para el índice de fechas se obtuvieron 1177 llaves distintas. El BST alcanzó una altura de 30, mientras que el RBT alcanzó una altura de 14. Para el índice de áreas se obtuvieron 879 llaves distintas. El BST alcanzó una altura de 20, mientras que el RBT alcanzó una altura de 13.

Por lo tanto, el RBT presentó una reducción importante en la altura:

- Fechas: de 30 a 14 niveles.
- Áreas: de 20 a 13 niveles.

Esto demuestra que el RBT es más resistente al orden de inserción y ofrece una estructura más equilibrada que el BST.

**b. ¿Percibe alguna diferencia entre la ejecución de los dos árboles (RBT y BST)? ¿Por qué pasa esto?**

Sí se observa una diferencia entre los tiempos de ejecución. En la medición realizada, el BST tardó aproximadamente **14455.53 ms**, mientras que el RBT tardó aproximadamente **14316.39 ms**.

En esta ejecución, el RBT fue ligeramente más rápido que el BST, con una diferencia aproximada de 139 ms. Sin embargo, la diferencia no es muy grande porque ambos árboles manejan la misma cantidad de llaves distintas y la mayor parte del tiempo de carga corresponde también a la lectura del archivo, la creación de listas y la actualización de los índices auxiliares.

El RBT realiza trabajo adicional durante las inserciones, porque debe ejecutar rotaciones y cambios de color. Estas operaciones tienen costo constante en cada nivel recorrido. Sin embargo, este costo adicional se compensa porque el RBT conserva una altura mucho menor.

En este caso, el BST alcanzó alturas de 30 niveles para fechas y 20 para áreas, mientras que el RBT alcanzó alturas de 14 y 13, respectivamente. Por esta razón, las búsquedas y actualizaciones en el RBT recorren menos nodos.

La diferencia de tiempo no es tan grande como la diferencia de alturas debido a que:

- Existen 319073 crímenes, pero solo 1177 fechas distintas y 879 áreas distintas.
- Muchas operaciones consisten en buscar una llave que ya existe y agregar el crimen a una lista.
- También se invierten recursos en procesar el archivo CSV y actualizar los mapas de ofensas.
- El RBT debe realizar rotaciones y cambios de color durante algunas inserciones.

En general, cuando el BST queda muy desbalanceado, el RBT puede ser más rápido y estable. Si el BST recibe datos en un orden favorable, ambos árboles pueden tener tiempos parecidos, aunque el RBT mantiene mejores garantías de rendimiento.

**c. ¿Existe alguna diferencia de complejidad entre los dos árboles (RBT y BST)?**

Sí existe una diferencia importante en el peor caso.

El costo de las operaciones principales depende de la altura del árbol. En un BST desbalanceado, la altura puede ser O(N). En un BST razonablemente balanceado, la altura promedio puede ser O(log n).

El RBT garantiza que la altura se mantenga en O(log n), independientemente del orden de inserción.

| **Operación** | **BST peor caso** | **BST caso promedio** | **RBT peor caso** |
|---|---|---|---|
| put | (O(N)) | (O(log N)) | (O(log N)) |
| get / contains | (O(N)) | (O(log N)) | (O(log N)) |
| get_min / get_max | (O(N)) | (O(log N)) | (O(log N)) |

| Operación | BST peor caso | BST caso promedio | RBT peor caso |
|---|---|---|---|
| keys / values por rango | (O(N)) | (O(log N + k)) | (O(log N + k)) |

Donde (k) representa el número de elementos encontrados dentro del rango.

En el BST, si las llaves se insertan en un orden ordenado o casi ordenado, el árbol puede degenerarse y las operaciones pueden tener costo (O(N)).

En el RBT, las rotaciones y los cambios de color garantizan que la altura permanezca logarítmica. Aunque cada inserción puede realizar trabajo adicional, dicho trabajo no cambia el orden de complejidad.

Para una carga de (M = 319073) crímenes y (N) llaves distintas, el costo de las operaciones sobre el índice es aproximadamente:

- BST en el peor caso: (O(MN)).
- RBT en el peor caso: (O(M log N)).

Por lo tanto, el RBT ofrece una garantía de rendimiento mucho mejor en el peor caso.

**d. ¿Existe alguna manera de cargar los datos en un árbol RBT de tal forma que su funcionamiento mejore?**

Sí. Aunque el RBT ya mantiene una altura balanceada, se puede mejorar el tiempo total de carga reduciendo el número de operaciones realizadas sobre el árbol.

Una primera estrategia consiste en agrupar previamente los crímenes por llave usando una tabla hash. De esta forma, primero se acumulan los crímenes por fecha y por área, y posteriormente se insertan en el RBT únicamente las llaves distintas.

En este caso, en lugar de realizar operaciones sobre el árbol para los 319073 crímenes, se realizarían aproximadamente:

- 1177 inserciones para las fechas.
- 879 inserciones para las áreas.

Otra optimización consiste en evitar operaciones innecesarias de búsqueda e inserción. Cuando una llave ya existe en el árbol, no es necesario volver a ejecutar put; basta con obtener la lista asociada y agregarle el nuevo crimen.

También se puede mejorar el proceso leyendo y procesando los datos en una sola pasada, manteniendo estructuras auxiliares para las fechas, áreas y tipos de ofensa.

Finalmente, si se dispone de todas las llaves antes de construir el índice, se pueden ordenar y construir un árbol balanceado de forma directa. Sin embargo, esta alternativa debe respetar las reglas de coloración del RBT, por lo que no basta simplemente con colorear todos los niveles de negro. La estrategia más sencilla y segura para este proyecto es agrupar primero los datos y luego insertar únicamente las llaves distintas en el RBT.

En conclusión, la mejora más práctica es:

1. Agrupar los crímenes por fecha y área.
2. Insertar en el RBT solo las llaves distintas.
3. Actualizar directamente las listas asociadas cuando la llave ya exista.
4. Evitar búsquedas y operaciones repetidas sobre el árbol.

Estas optimizaciones disminuyen el número de operaciones y reducen el tiempo total de carga sin cambiar la funcionalidad del índice.